# /automation

Automation service: /automation [list] | runs | run | help

## Overview

`/automation` is the TUI surface for the automation service (spec `openhands` T-016; FR-013, FR-014, FR-018, FR-033). An **automation** is a named, schedulable or webhook-triggered agent run with a durable definition and run history.

- **Webhook-triggered** automations fire from an inbound HTTP `POST /auto/<id>` (FR-013). The request body is supplied to the run as prompt context.
- **Schedule-triggered** automations fire when the scheduler reaches their due time, using the same schedule grammar as `/cron` (`every 2m`, `at <ts>`, …) (FR-014).
- Every run is **confined to the automation's configured execution backend** (FR-033), and every terminal run is recorded in durable run history with the automation id, trigger, start/end times, outcome, backend, and output reference (FR-018).

## Definitions

Automations are declared under the `automation` block in `.ragent/ragent.json`:

```
{
  "automation": {
    "enabled": true,           // master switch; absence leaves the service inert
    "run_history_cap": 500,    // run records retained per automation
    "scheduler_tick_secs": 30, // scheduler tick interval
    "automations": [
      {
        "id": "on-issue",              // unique; also the webhook URL segment
        "agent": "general",            // agent to run (defaults to defaultAgent)
        "prompt": "Triage {{payload}}", // {{payload}} is the trigger payload
        "trigger": { "kind": "webhook" },
        "backend": "local",            // execution backend the run is confined to
        "dispatch": [                  // optional third-party notifications
          { "kind": "slack", "url": "https://hooks.slack.com/...", "token_env": "SLACK_BOT_TOKEN"
        ]
      },
      {
        "id": "nightly",
        "trigger": { "kind": "schedule", "schedule": "every 2m" },
        "backend": "local"
      }
    ]
  }
}
```

The service is inert unless `automation.enabled` is explicitly `true` on a present `automation` block, so a project that never opted in starts no scheduler and accepts no webhook.

Dispatch targets name the environment variable their credential is read from (`token_env`); the value is

never stored in the config (FR-035). Supported kinds are `slack`, `github`, `linear`, `notion`, and `webhook` (a generic JSON POST). A target with no `url` is inert.

## Subcommands

| Subcommand | Description |
|---|---|
| `/automation` | List configured automations with trigger, schedule, backend, and next-due time |
| `/automation list` | Alias of `/automation` |
| `/automation runs <id>` | Show an automation's durable run history |
| `/automation run <id>` | Enqueue a manual run now |
| `/automation help` | Show the usage block |

## HTTP surface

| Method | Path | Auth | Purpose |
|---|---|---|---|
| `POST` | `/auto/<id>` | public | Webhook ingress; enqueues a run with the body as context (FR-013) |
| `GET` | `/automation` | bearer | List configured automations and next-due times |
| `GET` | `/automation/runs/<id>` | bearer | List an automation's run history |
| `POST` | `/automation/<id>/run` | bearer | Enqueue a manual run |

## CLI parity

```
ragent automation list
ragent automation runs <id>
ragent automation run <id>
ragent automation help
```

## Examples

```
# Webhook ingress (from an external system)
curl -s -X POST http://127.0.0.1:3000/auto/on-issue \
  -H "Content-Type: application/json" \
  -d '{"issue":"42","title":"fix login"}'

# Inspect from the TUI
/automation
/automation runs on-issue
/automation run nightly
```